Closures en Swift

420-249-AH - Programmation mobile iOS

1. Qu'est-ce qu'une closure ?

Une closure est un bloc de code que l'on peut :

- Stocker dans une variable
- Passer comme paramètre
- Exécuter plus tard

Une closure ressemble à une fonction, mais elle peut être déclarée sans nom.

En Swift, les closures sont très utilisées, notamment pour gérer des actions et des traitements.

2. Fonction vs closure

Une fonction classique :

```swift
func saluer(nom: String) {
    print("Bonjour \(nom)")
}

saluer(nom: "Alex")
```

La même logique sous forme de closure :

```swift
let saluer = { (nom: String) in
    print("Bonjour \(nom)")
}

saluer("Alex")
```

3. Syntaxe générale

```swift
{ (parametres) -> TypeRetour in
    // instructions
}
```

Le mot-clé in sépare :

Avant in	Après in
`Paramètres et type de retour`	Corps de la closure

4. Closure sans paramètre

```
let afficherMessage = {
    print("Bienvenue dans l'application")
}
```

Exécution :

```
afficherMessage()
```

5. Closure avec paramètres

```
let afficherUtilisateur = { (nom: String) in
    print("Utilisateur : \(nom)")
}
```

Utilisation :

```
afficherUtilisateur("Marie")
```

6. Closure avec valeur retournée

```
let additionner = { (a: Int, b: Int) -> Int in
    return a + b
}
```

Utilisation :

```
let resultat = additionner(10, 5)
print(resultat)
```

Résultat :

```
Console
15
```

7. Type d'une closure

La closure :

```
let additionner = { (a: Int, b: Int) -> Int in
    return a + b
}
```

possède le type :

```
(Int, Int) -> Int
```

On peut également écrire explicitement :

```swift
let additionner: (Int, Int) -> Int = {
    (a, b) in
    return a + b
}
```

8. Closure comme paramètre

Une fonction peut recevoir une closure.

```swift
func executerOperation(
    a: Int,
    b: Int,
    operation: (Int, Int) -> Int
) -> Int {
    return operation(a, b)
}
```

On peut ensuite écrire :

```swift
let resultat = executerOperation(
    a: 10,
    b: 5,
    operation: { (x, y) in
        return x + y
    }
)
```

Résultat :

```
Console
15
```

9. Trailing closure

Lorsque la closure est le dernier paramètre, Swift permet de l'écrire après les parenthèses.

Au lieu de :

```swift
executerOperation(
    a: 10,
    b: 5,
    operation: { (x, y) in
        return x * y
    }
)
```

on peut écrire :

```swift
executerOperation(a: 10, b: 5) { (x, y) in
    return x * y
}
```

Cette syntaxe est appelée Trailing Closure. Elle est très fréquente en Swift.

10. Syntaxe abrégée

Swift peut souvent déduire le type des paramètres.

```swift
let multiplier: (Int, Int) -> Int = {
    a, b in
    return a * b
}
```

Swift propose aussi :

```swift
$0
$1
$2
```

pour représenter les paramètres.

Exemple :

```swift
let multiplier: (Int, Int) -> Int = {
    $0 * $1
}
```

Utilisation :

```swift
print(multiplier(4, 5))
```

Résultat :

```
Console
20
```

11. Exemple — Action après connexion

```swift
func connecter(
    utilisateur: String,
    apresConnexion: () -> Void
) {
    print("Connexion de \(utilisateur)")
    apresConnexion()
}
```

Utilisation :

```
connecter(utilisateur: "Alex") {
    print("Ouverture de l'écran principal")
}
```

La closure est exécutée après la connexion.

12. Closure et capture de valeurs

Une closure peut utiliser des variables déclarées à l'extérieur.

```
var compteur = 0

let incrementer = {
    compteur += 1
    print(compteur)
}
```

Puis :

```
incrementer()
incrementer()
incrementer()
```

Résultat :

```
Console
1
2
3
```

On dit que la closure capture la variable compteur.

Cette notion deviendra importante lorsque nous étudierons ARC et les références.

13. Closures en SwiftUI

Les closures sont très présentes dans SwiftUI.

Exemple :

```
Button("Connexion") {
    print("Bouton appuyé")
}
```

Le bloc :

```
{
    print("Bouton appuyé")
}
```

est une closure.

```
Lorsque le bouton est appuyé
            ↓
   exécuter la closure
```

14. Exemple avec une condition

```swift
let verifierBatterie = { (niveau: Int) -> String in
    if niveau <= 20 {
        return "Batterie faible"
    }
    return "Batterie normale"
}
```

Utilisation :

```swift
print(verifierBatterie(15))
```

Résultat :

```
Console
Batterie faible
```

15. À retenir

Une closure est un bloc de code qui peut être stocké et exécuté plus tard.

Syntaxe :

```swift
let action = {
    print("Action")
}
```

Avec paramètres :

```swift
let calcul = { (a: Int, b: Int) -> Int in
    return a + b
}
```

Une closure peut aussi être passée à une fonction :

```swift
fonction {
    // closure
}
```

Et elle peut capturer des variables provenant de son environnement.

16. Exercice — Gestion d'actions dans une application

Créer une closure :

```
let afficherBienvenue
```

qui affiche :

```
Console
Bienvenue dans l'application
```

Créer ensuite une closure :

```
let calculerPourcentage
```

qui reçoit une valeur actuelle et une valeur totale, et retourne le pourcentage.

Exemple :

```
calculerPourcentage(50, 200)
```

doit retourner :

```
Console
25.0
```

Créer une fonction :

```
func executer(
    nomAction: String,
    action: () -> Void
)
```

Elle doit :

- Afficher le nom de l'action
- Exécuter la closure reçue

Exemple :

```
executer(nomAction: "Synchronisation") {
    print("Synchronisation terminée")
}
```

Résultat attendu :

```
Console
Action : Synchronisation
Synchronisation terminée
```

17. Exercice — Compteur d'actions

Créer une variable :

```
var nombreActions = 0
```

et une closure qui augmente cette valeur à chaque exécution.

Après trois appels :

```
nombreActions = 3
```